



Documentación Técnica de ALZA

Arquitectura, tecnologías, datos, seguridad, despliegue y puntos de extensión para developers.

Hecho por Ing. Juan Marengo
Versión 1.0 | Mayo 2026

Documento	Documentación Técnica de ALZA
Proyecto	ALZA Plataforma educativa inclusiva
Formato	PDF
Uso recomendado	Referencia interna para implementación, operación y entrega a clientes

1. Resumen técnico

ALZA es una plataforma educativa inclusiva construida como frontend web estático con JavaScript modular, Supabase como backend administrado y Vercel como destino de despliegue. La aplicación cubre home pública, autenticación por portal, dashboard de usuario, constructor administrativo de cursos, reproducción de lecciones, exámenes, progreso, planes y cuentas institucionales.

El diseño actual favorece simplicidad operativa: no hay bundler complejo, el build copia archivos estáticos a dist/ y la lógica de datos vive en módulos JavaScript que consumen Supabase directamente desde el navegador.

2. Stack y tecnologías

Capa	Tecnología	Uso en ALZA
Frontend	HTML, CSS, JavaScript ES Modules	Páginas estáticas, UI responsive, dashboards, course player y admin.
Backend	Supabase Auth, Database, Storage, RLS	Usuarios, roles, cursos, lecciones, progreso, archivos, planes y cuentas empresa.
Deploy	Vercel	Publicación estática usando buildCommand y outputDirectory dist.
Build local	Node.js	scripts/build-static.js copia HTML, public y src a dist.
Validación	scripts/validate-static.js	Revisa recursos faltantes y texto con encoding roto antes de deploy.

3. Mapa de archivos principales

Archivo	Responsabilidad
index.html	Home pública, navegación, secciones comerciales y entrada a autenticación.
auth.html / login.html / register.html	Flujo de selección de portal, login y registro.
platform.html	Dashboard de usuario final y empresa.
course.html	Vista de consumo de curso, lecciones, exámenes, progreso y descargas.
admin.html	Panel super admin, dashboard y gestión de cursos/contenido.
src/supabase-client.js	Inicialización de Supabase y utilidades compartidas.
src/session.js	Sesión, perfiles, rol, redirecciones y helpers de auth.
src/admin.js	CRUD de cursos, lecciones, exámenes, assets y dashboard admin.
src/platform.js	Dashboard cliente, cursos visibles por rol, progreso y cuentas institucionales.
src/course.js	Render del curso, bloqueo por avance, reproducción local, exámenes y progreso.
src/styles.css	Sistema visual completo de ALZA.
supabase/*.sql	Schema base, migraciones, seed de cursos y configuración de storage.

4. Modelo de datos

Tabla	Propósito
profiles	Perfil extendido de auth.users: rol, nombre, organización, ID de persona no oyente y etiqueta institucional.
plans	Planes activos por audiencia: oyente individual y empresa.
subscriptions	Registro de compra mock/estado activo por usuario.
company_seats	Cuentas creadas bajo una empresa: nombre, email, contraseña y etiqueta Nombre-Empresa.
courses	Cursos con título, slug, resumen, categoría, audiencia, cover, publicación y creador.
lessons	Lecciones y exámenes ordenados por posición, con video local, texto, preguntas y adjuntos.
enrollments	Inscripción del usuario al curso y progreso agregado.
lesson_progress	Estado completado/no completado por lección dentro de una inscripción.

5. Roles y acceso

- super_admin: acceso a admin.html, métricas, CRUD de cursos y contenido.
- empresa: compra plan empresarial y puede crear hasta 10 cuentas bajo la institución.
- persona_oyente: requiere plan individual para consumir cursos.
- persona_discapacidad_auditiva: registra un ID de 8 a 12 dígitos y no requiere compra de plan.

6. Flujo de contenido

- 1 El admin crea o edita un curso desde admin.html, definiendo título, resumen, categoría, audiencia y cover.
- 2 El admin agrega lecciones o exámenes, reordena bloques, sube videos locales y adjuntos descargables.
- 3 El contenido se guarda en Supabase: metadata en Database y archivos en Storage.
- 4 El usuario ve solo cursos publicados para su portal o rol.
- 5 course.html carga lecciones, bloquea las siguientes hasta completar las previas y recalcula el progreso.

7. Storage y archivos

Bucket / recurso	Uso esperado	Notas
course-videos	Videos locales de lección	Recomendado: MP4 H.264/AAC por compatibilidad web.
course-assets	Covers y adjuntos descargables	PDF, Excel, imágenes y otros documentos permitidos por política del bucket.
public/assets	Logos, paleta, GIF de bienvenida y recursos estáticos	Se copian a dist en el build.

8. Deploy y verificación

- 1 Configurar src/config.js con SUPABASE_URL y SUPABASE_ANON_KEY.
- 2 Ejecutar el SQL base y migraciones necesarias en Supabase.
- 3 Correr npm.cmd run build para generar dist/.

- 4 Correr `npm.cmd run validate:static` para detectar recursos faltantes o texto corrupto.
- 5 Subir commits a GitHub y conectar Vercel al repositorio.
- 6 En Vercel usar `buildCommand: npm run build` y `outputDirectory: dist`.

9. Consideraciones de mantenimiento

- Mantener `schema.sql` y migraciones alineados; no depender de datos hardcodeados para cursos reales.
- Cuando se agreguen nuevos tipos de archivo, revisar políticas y MIME types de Supabase Storage.
- No exponer service role keys en frontend; solo usar anon key con RLS.
- Probar cursos con los tres portales antes de compartir un link público.
- Antes de deploy cliente, ejecutar `build` y `validate:static`.